

## 72 Hour Build: Nightingale Candidate Brief

**Current State:** Your Electronic Health Record (EHR) holds structured, tabular snapshots, i.e. blood tests and are often incomplete/stale across visits and providers. Everything else that actually explains the patient's story (concerns, preferences, symptoms over time, follow-ups, context) is buried in free-text notes created per consult. Each clinician or staff member adds a new dated note, in their own format, in their own screen. The result is fragmented longitudinal history: there is no single shared narrative, no consolidated "what changed," or differential analysis, no reliable way to surface cross-visit context, and no lightweight collaboration across roles, just scrolling, searching by date, and guessing what matters.

**Goal:** Build a single, shared, longitudinal patient note clinic web application that enables real-time and longitudinal role-based collaboration across clinician, staff, patient-provided insights and AI-scribed notes from all patient-provider interactions. Surface a glanceable "most important or top card" for clinicians and staff with real insights. This is a communication and trust system, integrating patient-contributed insights with clinician and staff notes and tasks into one actionable, reliable record, augmented by intelligent AI summarization and prioritization.

**Reference Concepts (for inspiration):** Think of the collaborative power of **Google Docs/Sheets**, the structured data extraction of **Notion databases**, and the timeline-based history of **Slack threads** – all adapted to the stringent requirements of healthcare that feed into a structured knowledge note.

---

### Product Requirements

1. **Shared "Care Note":** a unified single page for each patient, designed for multi-role interaction:
  - "Top or Glance " display, optimized for rapid readability during a consult i.e. with content, open actions (e.g., "needs lab order," "waiting nurse follow-up"), and critical risk/flags. Must be fully readable and actionable in under 10 seconds.
  - **Longitudinal Timeline**
    - A time-ordered, continuous feed of all patient context to supplement the EHR.
    - Entries Include: Patient & AI session summaries/key questions (from AI-patient sessions), AI-scribed notes from consults (doctor-patient, nurse-patient), staff manual edits, clinician manual edits, and system-generated events.
    - **Metadata per Entry:** (example) author\_role (patient/staff/clinician/system), author\_id (or system if AI-generated), timestamp, type (session, consult, instruction, admin, etc.), provenance\_pointer (link to source message).
  - **Inline Collaboration**
    - Enable rich, collaborative editing and annotation within the notes i.e. think threaded comments (with resolve/unresolve states), optional mentions (@nurse\_name, @clinician\_name), and optional assignments ("Assign to staff"). This is up to your creativity.
  - **Revision History & Revert**

- Robust version control for all edits.
- Capabilities: Full version snapshots, "view changes since X" functionality, and the ability to revert to any previous version.
- Implementation: Store diffs or full snapshots (your architectural choice)

## 2. AI Scribe Integration & Smart Prioritization

- The system receives AI-scribed\_notes from three distinct interaction types:
  - **Doctor-Patient Consults:** Post-consult summary.
  - **Nurse-Patient Consults:** Post-consult summary.
  - **AI-Patient Sessions:** Pre- and post-consult patient interactions with the AI.
- These AI-scribed\_notes must appear as distinct entries from clinician or staff manual notes in the Longitudinal Timeline, clearly indicating their author\_role as system and their type (e.g., ai\_doctor\_consult\_summary, ai\_nurse\_consult\_summary, ai\_patient\_session\_summary).
- Provenance: Each AI-scribed note must include a provenance\_pointer to its original source (e.g., the specific session\_id for AI-patient interactions).
- BONUS: Self-Learning "Importance Logic"
  - Implement a mechanism for identifying and presenting critical information in the "Glance View," with an adaptive component.
  - Core Logic: Combine factors like recency, explicit risk\_level tags, tagged clinical entities (e.g., medications, chief complaint, allergies), and unresolved tasks.
  - Self-Learning Component: The system should learn from how clinicians and staff interact with the notes. If certain types of information (e.g., specific keywords, topics, or sections within AI-scribed\_notes or patient\_session\_summaries) are frequently *manually highlighted, edited, or commented upon* by clinicians, the importance logic should adapt over time to give higher priority to similar content in future suggestions.
  - Hard Constraint: Clinicians must be able to accept/reject highlight suggestions quickly. Each highlight must display a short risk\_reason and a provenance\_pointer to its source.
- BONUS: Hybrid Storage / Data Decay Logic: schema and logic for compressing older data

## 3. Role-Based Views + Permissions (RBAC)

Define and enforce access controls for various user roles: patient, staff, clinician, admin.

- **Access Rules (Minimum):**
  - **Patient:** Can view patient-facing summaries and instructions generated from the clinic web app notes; patient explicitly cannot view internal staff/clinician comments or raw AI-scribed notes.
  - **Staff:** Can view and add staff\_notes; cannot access other clinics' patient data.
  - **Clinician:** Can view and edit clinician\_sections; can view staff\_notes and all AI-scribed\_notes; access is clinic-scoped.
  - **Admin:** Clinic-scoped oversight across all patient data within their scope.
- Enforcement: Must be enforced server-side (e.g., via RLS, middleware, backend checks). UI-only checks are insufficient.

## 4. Provenance + Trust

Every piece of information, especially highlights, must be fully citable and traceable to its origin.

- Traceability: Clicking a highlight must navigate directly to its originating entry/span within the Longitudinal Timeline ("source of truth"), whether it's a manual note or an AI-scribed summary.
- Conflict Resolution: If clinician edits conflict with prior AI/patient memory (including AI-scribed notes), the clinician's entry takes precedence, OR **the system must flag the conflict for review**

## 5. Bonus: Ambient Consult capture

A system that allows both patients and clinical users to create AI-scribed entries directly from voice.

- Patient Voice Capture (available only in patient view) can record a conversation during consultation. The system can use PWA on mobile and must redact PHI before LLM processing, transcribe the recording and identify structured facts where appropriate and generate a patient consult session summary.
- Clinical or Staff Voice Capture (available only in clinical view) can record a clinician-patient or nurse-patient consult conversation through PWA on mobile or laptop and produce speaker labelled transcript, timestamps, confidence markers, code-switching support, clinical summary, provenance back to source segments. Extra bonus credit if you thoughtfully build for use in noisy environments, diarization, overlap handling, multilingual medical terminology, or multi-device capture

---

## Technical Constraints and Architecture:

**Access Control:** RBAC: Enforced server-side. Patient cannot access Clinician. Clinicians cannot overwrite Staff notes. Staff cannot overwrite Clinician notes.

**Latency:** P95 for loading the "consult glance view" must be  $\leq 300\text{ms}$  on a warm path (state how you measured/approximated this in your brief).

**Privacy and Security:** Synthetic Data Only. Include a No PHI Redaction Pipeline: You must redact names, IC/ID numbers, and phones *before* sending text to the LLM. TLS in transit + encryption at rest.

**Tech Stack:** Choose what gives you speed to bring idea to execution. (Python/Node suggested). Any LLM.

---

Required Micro-Tests: Include automated tests and how to run tests steps:

- test\_rbac\_scope.py
  - Assert that staff and clinicians cannot write or edit notes as each other.
  - Assert that a patient cannot access internal comments or raw AI-scribed notes.
- test\_revision\_history.py
  - Assert that editing a note increments its version.
  - Assert that reverting returns content to a prior state.
  - Assert that the audit log shows who changed what (metadata only).
- test\_highlight\_provenance.py
  - Generate highlights (including from AI-scribed notes).

- Assert each highlight has a `provenance_pointer` that successfully resolves to an entry/span in the timeline.
  - `test_concurrent_edits.py`
    - Demonstrate that two roles editing different sections concurrently do not overwrite each other's changes.
    - If conflicts occur on the same section, demonstrate a deterministic resolution strategy.
  - BONUS: `test_self_learning_importance.py`
    - Simulate manual interaction (e.g., pinning a highlight from an `AI-scribed_note`).
    - Assert that subsequent highlight suggestions (for similar content) demonstrate increased priority based on this simulated learning. (test can be conceptual, describing expected outcome).
- 

## Deliverables

1. Git Repository:
  - Working application.
  - Automated tests (as specified above).
  - Clear commit history.
2. README with setup & run instructions, where redaction happens, how you enforce RBAC.
3. 2–3 Page Technical Brief:
  - Architectural diagram and explanation.
  - Comprehensive data schema (show how Entries ↔ Comments ↔ Versions ↔ Highlights ↔ Provenance ↔ `AI_Scribed_Notes` are linked, and how/if learning mechanism integrates).
  - Assumptions, first-principles thinking, and trade-offs/scope decisions.
4. **ATTRIBUTION.txt:** List all external libraries, models, and their licenses.
5. Demo Video: Clearly demonstrate the chosen scenarios.

## Recommended Demo Scenarios

### Scenario A – “Glance View in Action & AI Scribe Integration”

- Staff opens a patient page and immediately sees the “Top Card” in under 10 seconds.
- Interaction: Click a highlight sourced from an `AI-scribed_note` to jump to its exact entry in the timeline.

### Scenario B – “Collaborative Audit Trail & Importance Learning”

- Staff adds a new note and adds a comment with an i.e. `@clinician` tag.
- Clinician Action: Clinician *manually highlights* a specific phrase within an `AI-scribed_note` and also edits a section of the patient's plan.
- Audit: Demonstrate the revision history, showing diffs, and perform a revert to a previous state.

### Scenario C – “Longitudinal Context”

- History View: Showcase entries from different dates (e.g., April 15, 2025, and Feb 6, 2026), including a mix of manual and AI-scribed notes.
  - Highlight Logic: Explain how highlights prioritize recent, unresolved actions, and clinician-confirmed items over older, less relevant data, with a specific focus on how the self-learning component influences this.
  - BONUS: Demonstrate (or explain architecturally) your approach to data decay for older, less critical entries.
- 

## Scoring (20 points)

- **Glanceability & Actionability (6)**: How effectively does the consult view provide immediate value? Are highlights relevant and actionable? Would the clinician or staff be overwhelmed by it?
  - **Collaboration & AI Integration (5)**: Robustness of comments, role-based interaction, revision control, and seamless integration and display of AI-scribed\_notes.
  - **Provenance & Trust (4)**: Clarity of citations (for both manual and AI-generated content), conflict handling, and confidence in the information's origin.
  - **Security & Privacy (3)**: Real-world RBAC implementation, strict PHI redaction (for all data streams), and clean logs.
  - **Communication (2)**: Conciseness of the brief, clarity of the demo, and explicit discussion of trade-offs.
  - **Bonus Nightingale Alignment: (10)**: Hybrid Storage / Data Decay Policy, Self-Learning Implementation
- 

## Timeline & submission

- **Due: Friday, August 28 2026, 5:30 PM SGT/MYT**
- **Submit:** email your **repo link (or zip)**, **brief** and **deliverables** to [irakumar@ntngale.com](mailto:irakumar@ntngale.com), (cc [frank.ng@ntu.edu.sg](mailto:frank.ng@ntu.edu.sg), [carrene.teo@ntu.edu.sg](mailto:carrene.teo@ntu.edu.sg)), **Subject:** Nightingale 72HR Build – <Your Name>

## Tools & data

- Use any resources (ChatGPT, coding copilots, open-source models, etc). Your presentation, creativity, and brief will give us strong clues about how much you care. If this problem *doesn't* pull you in, go solve another problem.
- Can use synthetic data sets.
- Don't build a generic Notion page. Think about the psychology behind the build. We trust LLMs but up to a point and then we need reassurance from clinicians/staff. How do we build a system for that?

*Candidates pro-tip: Focus on the core problems and build a working prototype that is actually safe and useful. Ask yourself: Clarity and relentlessness beat polish. If you get stuck, keep asking questions and keep building. A great build has your purest intention and is a gift to the universe.*

*\*\* In 48 hours or so, we may add a hint here to help you benchmark your build quality.*